

Week-5

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Answer:

Traditional MapReduce processes data by reading and writing to disk after every step, which makes it slower, especially when working with large datasets. It is not suitable for iterative tasks like machine learning because the data has to be loaded from disk repeatedly. Writing MapReduce programs is also more complex and requires more code.

Apache Spark overcomes these limitations by processing data in memory, which makes execution much faster. It provides simple APIs for Python, Java, Scala, and R, making development easier. Spark also supports SQL queries, streaming, machine learning, and graph processing within the same framework. Because of its speed, ease of use, and support for multiple data processing tasks, Spark is the preferred choice for modern big data applications.

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

Answer:

Spark uses in-memory computing, which means it stores intermediate data in RAM instead of writing it to disk after every operation. This allows Spark to reuse the same data multiple times without reading it again from storage.

In machine learning algorithms, the same dataset is processed repeatedly during training. In traditional disk-based systems like MapReduce, the data is read from and written to disk in every iteration, which increases execution time. Spark avoids this overhead by keeping the data in memory, making iterative algorithms much faster and more efficient. This is one of the main reasons why Spark is widely used for machine learning and big data analytics.

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

Answer:

```
Python 2 minutes ago (1s) 3
```

```
# Remove duplicate rows based on user_id and transaction_date
df_clean = df.dropDuplicates(["user_id", "transaction_date"])

# Display the cleaned DataFrame
df_clean.show()
```

> See performance (1)

df_clean: pyspark.sql.connect.dataframe.DataFrame = [user_id: long, transaction_date: string ... 7 more fields]

	user_id	transaction_date	region	product_category	sale_amount	city	subscription	age	status
	101	2024-01-01	West	Furniture	500	Mumbai	Premium	25	Completed
	102	2024-01-02	East	Technology	700	Pune	Standard	32	NULL
	103	2024-01-03	West	Office Supplies	NULL	Delhi	Premium	28	
	104	2024-01-04	South	Technology	900	Mumbai	Standard	19	Completed

Q4: Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

Answer:

```
from pyspark.sql.functions import avg

df.filter(df.region == "West") \
    .groupBy("product_category") \
    .agg(avg("sale_amount").alias("Average_Sale")) \
    .show()
```

> [See performance \(1\)](#) ✓ Checked for improvements

product_category	Average_Sale
Furniture	500.0
Office Supplies	NULL

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

Answer:

The `.na.drop()` function removes rows that contain null values, whereas `.na.fill()` replaces null values with a specified value. In this example, all null values in the status column are replaced with **"Unknown"** instead of removing the records.

```
# Fill null values in the status column with 'Unknown'
df_fill = df.na.fill({"status": "Unknown"})

# Display the DataFrame
df_fill.show()
```

> [See performance \(1\)](#)

> `df_fill: pyspark.sql.connect.dataframe.DataFrame = [user_id: long, transaction_date: string ... 7 more fields]`

user_id	transaction_date	region	product_category	sale_amount	city	subscription	age	status
101	2024-01-01	West	Furniture	500	Mumbai	Premium	25	Completed
102	2024-01-02	East	Technology	700	Pune	Standard	32	Unknown
101	2024-01-01	West	Furniture	500	Mumbai	Premium	25	Completed
103	2024-01-03	West	Office Supplies	NULL	Delhi	Premium	28	
104	2024-01-04	South	Technology	900	Mumbai	Standard	19	Completed

Q6: Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Answer:

```
from pyspark.sql.functions import count

df.groupBy("city") \
  .agg(count("*").alias("Total_Records")) \
  .filter("Total_Records > 100") \
  .show()
```

city	Total_Records
------	---------------

Q7: How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Answer:

Spark DataFrames are immutable, which means the original DataFrame cannot be changed after it is created. Whenever we perform operations like dropping a column, renaming a column, or filtering data, Spark creates a new DataFrame instead of modifying the existing one. Therefore, we usually assign the result to a new DataFrame or overwrite the existing variable with the transformed DataFrame.

Q8: Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Answer:

```
df.filter(
  (df.age.between(18, 30)) &
  (df.subscription == "Premium")
).show()
```

> [See performance \(1\)](#)

user_id	transaction_date	region	product_category	sale_amount	city	subscription	age	status
101	2024-01-01	West	Furniture	500	Mumbai	Premium	25	Completed
101	2024-01-01	West	Furniture	500	Mumbai	Premium	25	Completed
103	2024-01-03	West	Office Supplies	NULL	Delhi	Premium	28	

Q9: When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?

Answer:

Null values can affect the accuracy of mathematical calculations such as `sum()` and `avg()`. If null values are not handled properly, the results may be incorrect or incomplete. Replacing or removing null values before performing aggregations helps produce more reliable and meaningful results.

Q10: Write the code to **revise** a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

Answer:

```
from pyspark.sql.functions import lit
from pyspark.sql.types import TimestampType

# Create a sample raw_timestamp column
df_timestamp = df.withColumn("raw_timestamp", lit("2024-01-01 10:30:00"))

# Cast it to TimestampType and rename it
df_timestamp = df_timestamp.withColumn(
    "event_time",
    df_timestamp.raw_timestamp.cast(TimestampType())
).drop("raw_timestamp")

df_timestamp.show()
```

> [See performance \(1\)](#)

> df_timestamp: pyspark.sql.connect.dataframe.DataFrame = [user_id: long, transaction_date: string ... 8 more fields]

user_id	transaction_date	region	product_category	sale_amount	city	subscription	age	status	event_time
101	2024-01-01	West	Furniture	500	Mumbai	Premium	25	Completed	2024-01-01 10:30:00
102	2024-01-02	East	Technology	700	Pune	Standard	32	NULL	2024-01-01 10:30:00
101	2024-01-01	West	Furniture	500	Mumbai	Premium	25	Completed	2024-01-01 10:30:00
103	2024-01-03	West	Office Supplies	NULL	Delhi	Premium	28		2024-01-01 10:30:00
104	2024-01-04	South	Technology	900	Mumbai	Standard	19	Completed	2024-01-01 10:30:00

Q11: Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Answer:

A shuffle is the process of redistributing data across different partitions so that records with the same key are grouped together. During operations like `groupBy()`, Spark moves data between executors, which increases network communication and execution time. Since data is transferred across partitions, `groupBy()` is considered a wide transformation.

Q12: Write a code snippet that identifies and **removes** rows where the email column contains null values OR the username is an empty string.

Answer:

```
from pyspark.sql.functions import lit, when, col

# Add sample columns
df_email = df.withColumn("email", lit("user@example.com")) \
    .withColumn("username", lit("user123"))

# Create sample null and empty values
df_email = df_email.withColumn(
    "email",
    when(col("user_id") == 102, None).otherwise(col("email"))
)

df_email = df_email.withColumn(
    "username",
    when(col("user_id") == 103, "").otherwise(col("username"))
)

# Remove rows
df_clean = df_email.filter(
    col("email").isNotNull() &
    (col("username") != "")
)

df_clean.show()
```

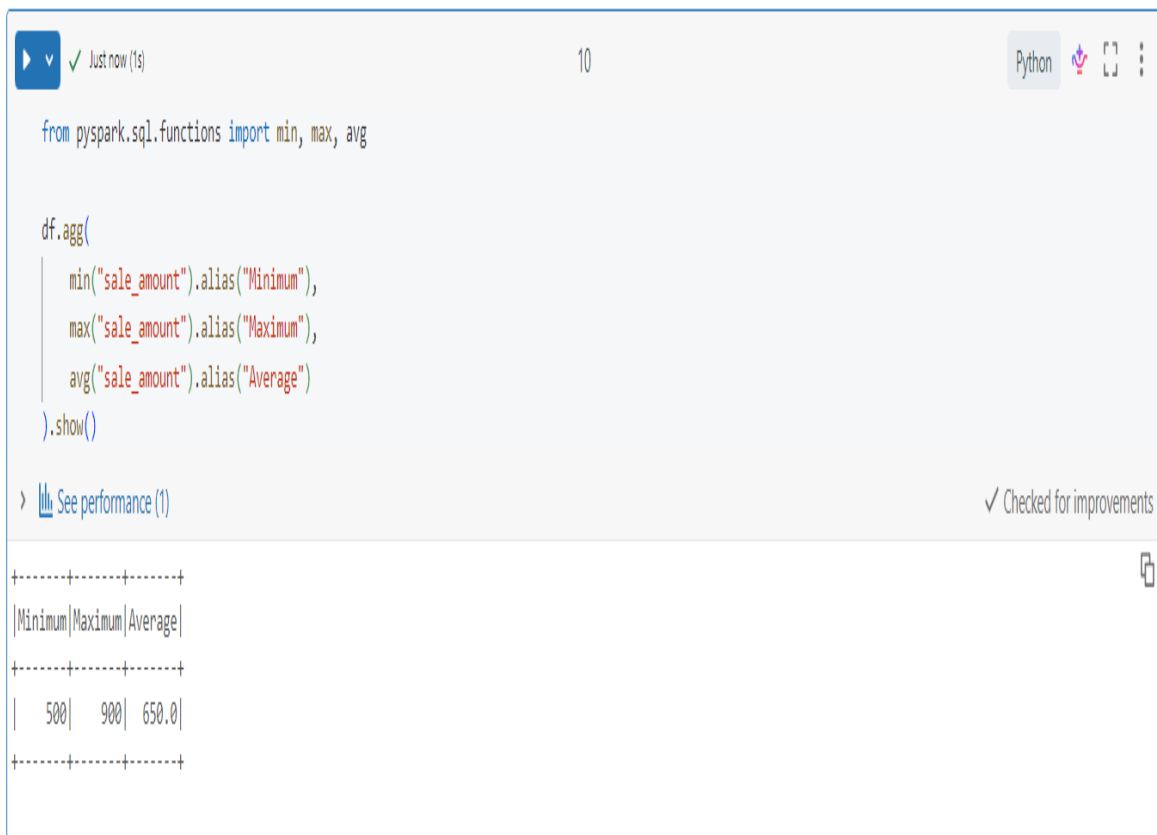
> See performance (1) ✓ Checked for improvements

df_email: pyspark.sql.connect.dataframe.DataFrame = [user_id: long, transaction_date: string ... 9 more fields]
df_clean: pyspark.sql.connect.dataframe.DataFrame = [user_id: long, transaction_date: string ... 9 more fields]

user_id	transaction_date	region	product_category	sale_amount	city	subscription	age	status	email	username
101	2024-01-01	West	Furniture	500	Mumbai	Premium	25	Completed	user@example.com	user123
101	2024-01-01	West	Furniture	500	Mumbai	Premium	25	Completed	user@example.com	user123
104	2024-01-04	South	Technology	900	Mumbai	Standard	19	Completed	user@example.com	user123

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Answer:



```
from pyspark.sql.functions import min, max, avg

df.agg(
    min("sale_amount").alias("Minimum"),
    max("sale_amount").alias("Maximum"),
    avg("sale_amount").alias("Average")
).show()
```

> [See performance \(1\)](#) ✓ Checked for improvements

Minimum	Maximum	Average
500	900	650.0

Q14: In the context of **cleaning a dataset**, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

Answer:

When `inferSchema=true` is used, Spark automatically determines the data type of each column. If a date column contains inconsistent or invalid date formats, Spark may assign the wrong data type or treat the column as a string. This can lead to incorrect results, errors during processing, or problems while performing date-related operations. Therefore, it is important to validate and clean the data before relying on schema inference.

Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by `store_id` to calculate total revenue.

Answer:

```
from pyspark.sql.functions import lit, sum

# Create sample columns
df_pipeline = df.withColumn("store_id", lit(1)) \
    .withColumn("price", df.sale_amount)

# Step 1: Remove duplicates
df_pipeline = df_pipeline.dropDuplicates(["user_id", "transaction_date"])

# Step 2: Fill null prices with 0
df_pipeline = df_pipeline.na.fill({"price": 0})

# Step 3: Group by store_id and calculate total revenue
result = df_pipeline.groupBy("store_id") \
    .agg(sum("price").alias("Total_Revenue"))

result.show()
```

> [See performance \(1\)](#)

> df_pipeline: pyspark.sql.connect.dataframe.DataFrame = [user_id: long, transaction_date: string ... 9 more fields]

> result: pyspark.sql.connect.dataframe.DataFrame = [store_id: integer, Total_Revenue: long]

store_id	Total_Revenue
1	2100